Wind Farm Layout Optimization

A First Metaheuristics Exercise

MetaLab Suite

Problem #1 — beginner edition

October 10, 2026

Abstract

This is the friendly, first-days-of-the-course version of the Wind Farm Layout Optimization (WFLO) problem. We strip the problem down to a small grid, two wind directions, a single wake rule and one objective, so that the only thing left to do is the thing we actually want to teach: *how to design a metaheuristic*. By the end you will have implemented your first hill climber and compared it against random search and the exact optimum. The full, continuous version of the problem is described in the advanced companion document.

Contents

1 The problem in words **1**

2 A simple model **1**
2.1 Domain and decision variables 1
2.2 The wake rule 2
2.3 Turbine power 2
2.4 Objective: expected total power 2

3 Why we need a metaheuristic **3**
3.1 The search space is exponential 3
3.2 The objective is a nasty, non-smooth landscape 3

4 Your first metaheuristic: hill climbing **3**
4.1 A neighbourhood for this problem 3

5 Your task **4**

6 Further reading **4**

1 The problem in words

A wind farm is a small **grid of cells**. We own N turbines and must decide **which** N **cells** to occupy. The catch is the *wake effect*: a turbine slows the air behind it, so any turbine standing downwind of it receives less wind, and therefore produces less power. Placing turbines far apart reduces this interference, but we cannot make the farm infinitely large.

To make the problem concrete and small enough to reason about, we use a "toy" model with exactly two wind directions:

- the wind blows **east** (left $\rightarrow$ right) with probability p_{east};

- the wind blows **south** (top → bottom) with probability $p_{\text{south}} = 1 - p_{\text{east}}$.

The two directions are what make the problem interesting. With only the east wind, the best plan would be to line every turbine up in the *leftmost column* (nothing is upwind of anything). But the south wind would then knock those very turbines down: standing in a single column, they shelter one another vertically. The optimal layout must **stagger** the turbines so that the wake losses are shared fairly between the two directions.

2 A simple model

2.1 Domain and decision variables

The farm is an $m \times m$ grid of cells. Number the cells $0, 1, \ldots, m^2 - 1$; cell c sits at row and column

$$(r, y) = (c \div m,\ c \bmod m). \tag{1}$$

A **layout** is a set

$$X = \{c_1, c_2, \ldots, c_N\} \subset \{0, 1, \ldots, m^2 - 1\}, \qquad |X| = N, \tag{2}$$

i.e. a choice of N distinct cells. This set of cells is the only decision we make.

2.2 The wake rule

Wind direction determines the direction of the wakes, and therefore which turbines are "upwind" of which. We use one fixed loss factor $\lambda_{\text{wake}} = \texttt{wake_loss} \in [0, 1]$ and the simple rule:

> A turbine loses a fraction λ_{wake} of its wind speed *for every turbine that is upwind of it on its own line.*

"Upwind on its own line" means, concretely,

- **east wind:** other turbines in the *same row* with a *smaller column*;
- **south wind:** other turbines in the *same column* with a *smaller row.*

Let k_i^d be the number of turbines upwind of turbine i for direction $d \in \{\text{east},\ \text{south}\}$. The speed turbine i sees is

$$u_i^d = u_\infty \left(1 - \lambda_{\text{wake}}\right)^{k_i^d}, \tag{3}$$

where $u_\infty = \texttt{free_speed}$ is the free-stream wind speed. Being second on a line costs one factor $(1 - \lambda_{\text{wake}})$, being third costs two, and so on.

2.3 Turbine power

Each turbine converts wind speed into electrical power through the cubic power curve

$$P(u) = P_r \left(\min\!\left(1,\ \tfrac{u}{u_r}\right)\right)^3, \tag{4}$$

where $u_r = \texttt{rated_speed}$ and $P_r = \texttt{rated_power}$ are the rated speed and rated power. Below the rated speed the power grows with the cube of the wind ($P \propto u^3$); above it the turbine is capped.

2.4 Objective: expected total power

For one direction, the farm produces the sum of its turbines' power,

$$P^d(X) = \sum_{i=1}^{N} P\Big(u_i^d(X)\Big)\,, \tag{5}$$

and the quantity we want to **maximise** is the expected power over the two directions:

$$\boxed{P_{\text{total}}(X) = p_{\text{east}}\, P^{\text{east}}(X) + p_{\text{south}}\, P^{\text{south}}(X) = \sum_{d} p_d \sum_{i=1}^{N} P\Big(u_i^d(X)\Big)\,.} \tag{6}$$

The complete problem is

$$\max_{X}\ P_{\text{total}}(X) \quad \text{subject to} \quad X \subset \{0,\dots,m^2-1\},\ |X| = N. \tag{7}$$

> **Constraint (only one).** The N turbines occupy *distinct* cells. Everything else is objective.

3 Why we need a metaheuristic

3.1 The search space is exponential

The number of possible layouts is the number of ways to choose N cells out of m^2:

$$|\mathcal{X}| = \binom{m^2}{N}. \tag{8}$$

For the default instance ($m = 4$, $N = 6$) this is only $\binom{16}{6} = 8008$ — small enough that we *can* check every layout by brute force, which is wonderful for grading. But the space explodes: going to $m = 8$, $N = 16$ gives $\binom{64}{16} \approx 4.9 \times 10^{14}$ layouts, far beyond enumeration.

3.2 The objective is a nasty, non-smooth landscape

Even though the problem looks tame, the objective (6) has two features that defeat simple, greedy reasoning:

- **coupling.** Whether a turbine suffers a wake depends on *which other* turbines were placed in its row and column, so the contribution of a turbine cannot be optimised in isolation.
- **multimodality.** Because the two directions pull in different ways, good layouts come in competing "families" (staggered patterns aligned one way or the other), separated by mediocre layouts. There are many local optima.

Trying to improve a layout one greedy step at a time gets trapped in whichever local optimum it happens to fall into. This is exactly the situation *metaheuristics* are built for.

4 Your first metaheuristic: hill climbing

A *trajectory-based* metaheuristic walks a single solution through the search space. Hill climbing is the simplest one:

1. start from a random layout;

2. make a small random change to get a **neighbouring** layout;

3. if the neighbour is better, keep it; otherwise throw it away;

4. repeat many times, remembering the best layout ever seen.

Two ingredients define any such method: the **neighbourhood** (what a "small change" is) and the **acceptance rule** (when to keep the change). For hill climbing the acceptance rule is simply: *keep it if it is better*. The neighbourhood is the part we leave to you.

4.1 A neighbourhood for this problem

A natural move is to take **one turbine** and move it to a **currently empty** cell:

$$\text{neighbour}(X): \quad \text{pick } i \text{ at random}, \quad \text{set } c_i \leftarrow c \text{ with } c \notin X. \tag{9}$$

This always produces a valid layout (still N distinct cells), which is one of the reasons this encoding is so pleasant.

Algorithm 1 Hill climbing (maximisation)

$X \leftarrow \textsc{RandomLayout}(m, N)$
$X^\star \leftarrow X$ ▷ best layout so far
for $t = 1$ **to** T **do**
  $X' \leftarrow \textsc{Neighbour}(X)$ ▷ the move you implement
  if $P_{\text{total}}(X') > P_{\text{total}}(X)$ **then**
    $X \leftarrow X'$
    if $P_{\text{total}}(X) > P_{\text{total}}(X^\star)$ **then**
      $X^\star \leftarrow X$
    end if
  end if
end for
return $X^\star$

5 Your task

Everything is provided except the neighbourhood. Open `hill_climbing.py` and implement `random_neighbour()`: move one random turbine to a random empty cell and return the new layout. Then:

1. run hill climbing and compare its best power with random search;

2. compare both with the exhaustive optimum (`model.brute_force`);

3. read the suggestions below and try one of them.

Table 1: Where each piece lives (folder `starter/`).

File	Role
`model.py`	given: instance, wake rule, objective, brute force
`generate.py`	given: build instances (e.g. $m = 4$, $N = 6$)
`random_search.py`	given: baseline you must beat
`hill_climbing.py`	**your task**: implement the neighbourhood
`app.py`	given: interactive grid visualisation

Stretch goal — simulated annealing. Hill climbing only ever accepts improvements, so it stops as soon as no neighbour is better. **Simulated annealing** instead accepts a worsening move with probability $\exp(-\Delta/T)$, where $\Delta > 0$ is how much worse the neighbour is and T is a *temperature* that slowly decreases. Early on it wanders freely (high T); later it behaves like hill climbing (low T). Changing the acceptance rule in Algorithm 1 to this rule is a one-line experiment with a very visible effect on the convergence curve.

6 Further reading

The advanced companion document derives the continuous, physically detailed version of this problem, including the Jensen wake model, the full power curve, topographical constraints, and the mathematical justification for using trajectory metaheuristics. The classical references are [1] for the wake model and [2, 3] for early metaheuristic approaches to turbine placement. Simulated annealing is due to [4].

References

[1] N. O. Jensen, "A note on wind generator interaction," Risø National Laboratory, Roskilde, Denmark, Tech. Rep. Risø-M-2411, 1983.

[2] G. Mosetti, C. Poloni, and B. Diviacco, "Optimization of wind turbine positioning in large wind farms by means of a genetic algorithm," *Journal of Wind Engineering and Industrial Aerodynamics*, vol. 51, no. 1, pp. 105–116, 1994.

[3] S. A. Grady, M. Y. Hussaini, and M. M. Abdullah, "Placement of wind turbines using genetic algorithms," *Renewable Energy*, vol. 30, no. 2, pp. 259–270, 2005.

[4] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, "Optimization by simulated annealing," *Science*, vol. 220, no. 4598, pp. 671–680, 1983.